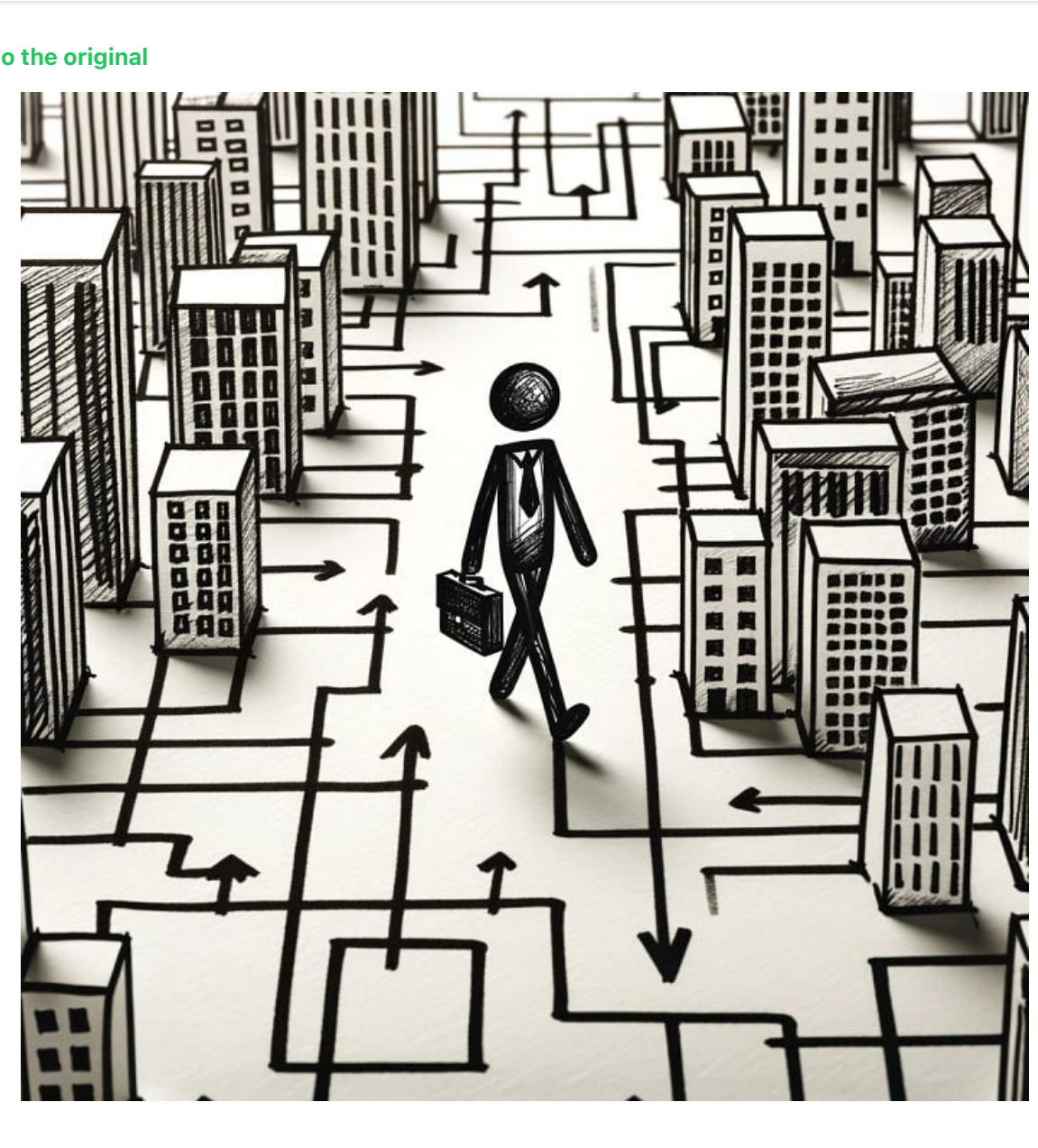
< Go to the original



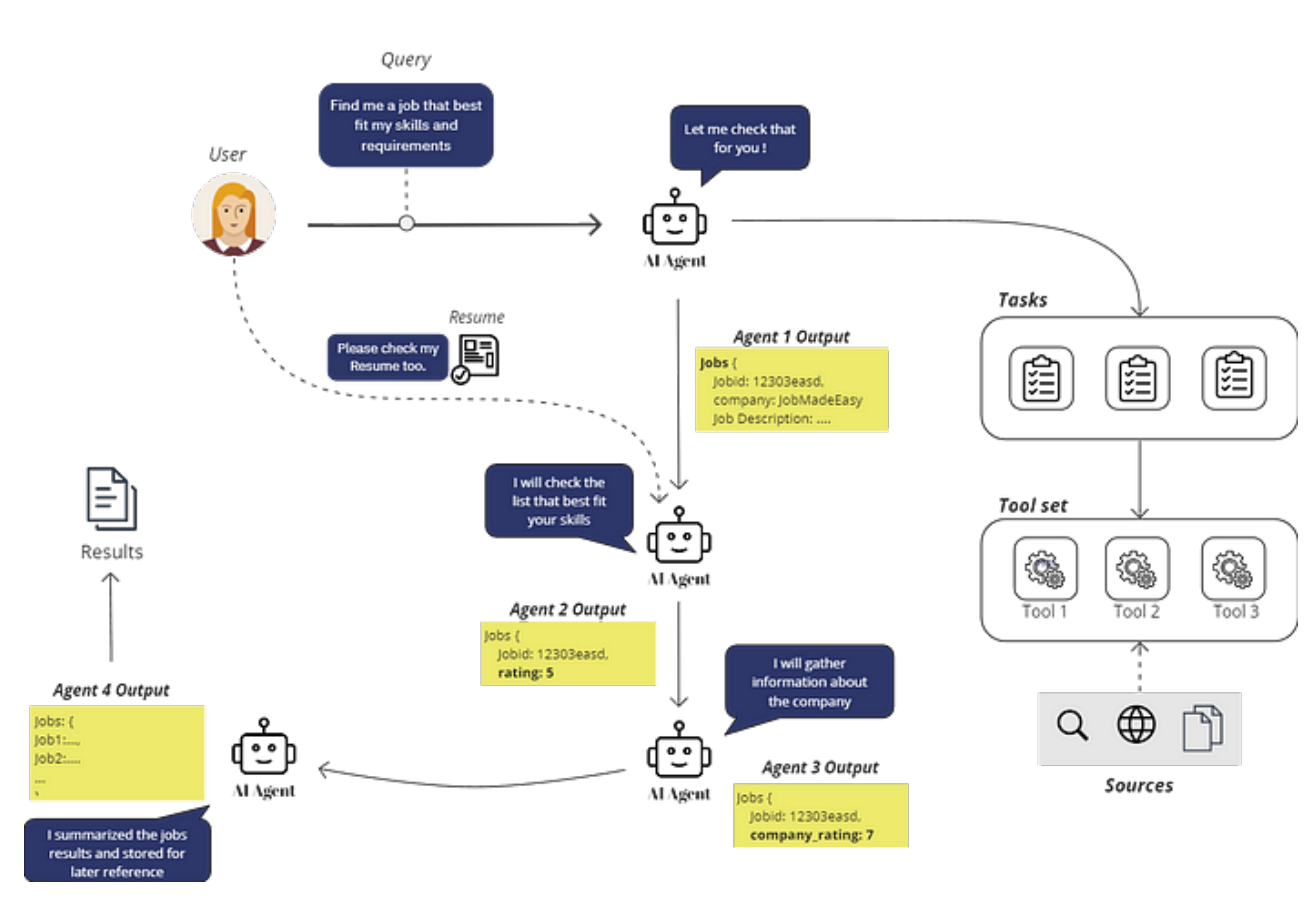
Automate Your Job Search with AI Agents

A step-by-step guide on building a team of AI agents that automate and refine the search and selection process matching job seekers'...

Hussein Jundi
Follow

Data Science Collective a11y-light ~18 min read · February 17, 2025 (Updated: February 17, 2025) ·
Free: No

A step-by-step guide on building a team of AI agents that automate and refine the search and selection process matching job seekers' skills with job requirements



Job Search with AI Agents — Image by Author

> User: AI find the best jobs that fit my profile. AI: On it..

Table of Content

Intro

1. Common challenges impacting every job seeker (The Why..)
2. AI Agent's crucial role in optimizing the job search process (The What..)
3. Introducing AI Agents capabilities to the process (The How..)

Hands-on: Implementing AI-powered job search engine

1. Project Structure
2. Framework
3. Tools
4. Data
5. Tasks
6. Agents
7. LLM
8. Output Model
9. Putting Everything Together: The Crew
10. Result Analysis

Source Code Summary Potential Improvements References

Job search can be difficult and time-consuming. It can take weeks and up to several months to navigate through job listings until eventually landing a job.

Searching for 5 months
100s.. of Applications

Job Seeker Struggles — Image by Author

The amount of time depends on many factors, such as expertise, role demand, markets, etc.. but according to unemployment data (including those passively scouting) from the U.S. Bureau of Labor Statistics shows that the median duration in March 2024 was **21.6 weeks ~ 5 months**.

On a human age scale, and considering the amount of time an average job seeker takes to search for new jobs, that's a lot of unretrievable lifetime just spent on screening 100s of applications.

The number of applications submitted is one of the success factors for landing a job. A study result positioned the average number of applications it takes to land the right position as **100–200+ applications**, which nonetheless can have outliers depending on the market, economic situation, and an applicant's expertise. The sheer amount of applications and effort one puts in searching for a job justifies our motivation behind building efficient and scalable solutions saving individuals thousands of hours.

Common challenges facing every job seeker (The Why..)

Struggles of Job Search — Image generated by Author using DALLE-3 from OpenAI

Finding the role that best fits one's skills and requirements in a market flooded with job listings is challenging. If you've been in such a position you know how stressful and tiring it can be going through hundreds of listings trying to crunch and match skills, salary ranges, prerequisite requirements, etc... The longer the process takes the lower the motivation one has and the higher their chances to quit the search and settle for less.

The time it takes to find the right role varies depending on one's experience, application timing, and how in-demand their skills are in the job market. Other uncontrollable factors like those impacting the industry and organizations they're applying for.

The challenges we tackle in building an AI-powered job searching engine in this article are mainly focused on optimizing the job search and the matching process, thus resolving key challenges like the following:

Key Challenges

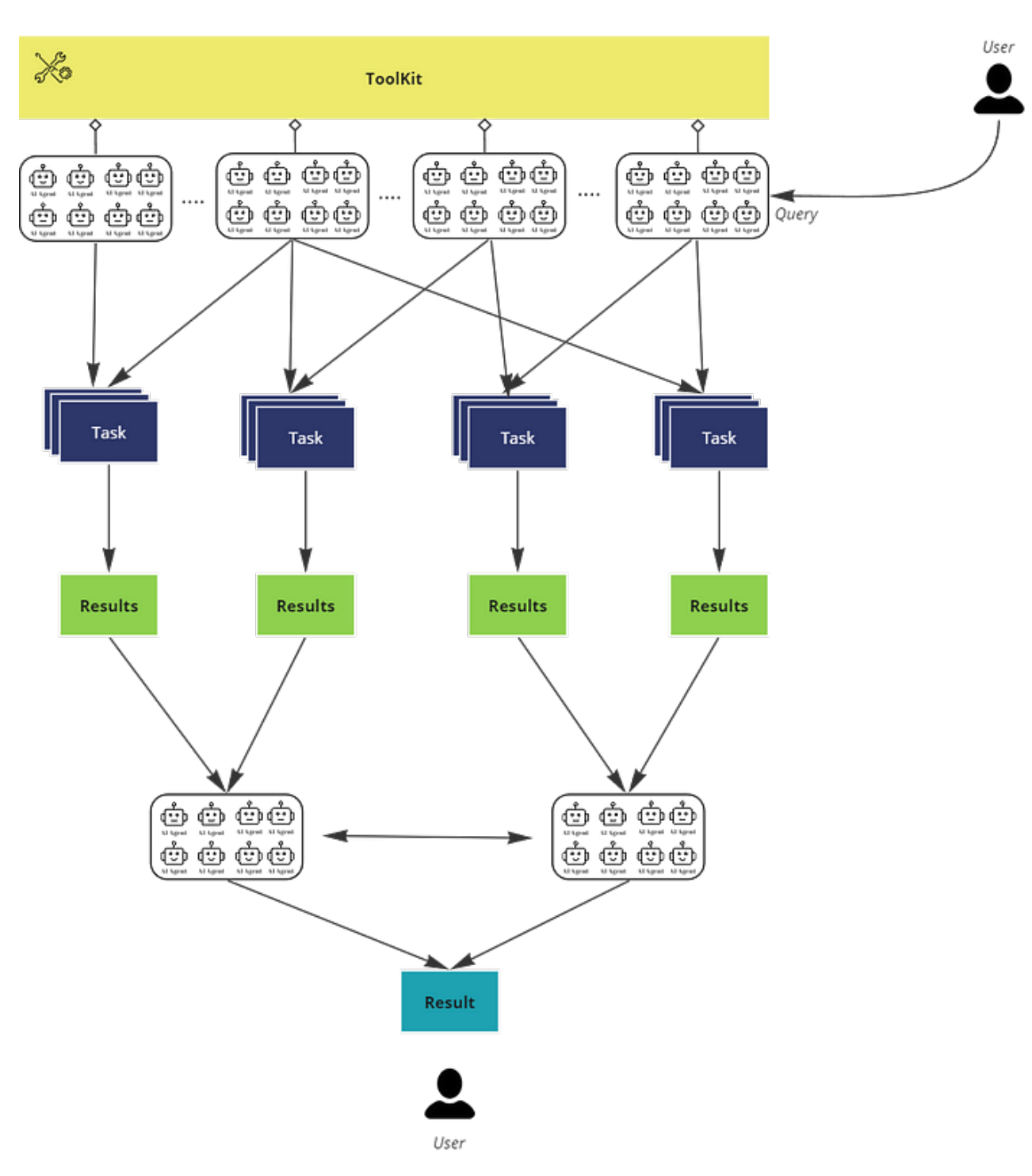
- **Job Advertisements Search:** Traditionally, screening jobs begins with choosing the right source platform where jobs are advertised. Filtering through jobs with specific criteria and evaluating the final result by digesting all their content. For better reachability, many choose to work on more than one platform. Hardships start when one has to juggle between multiple sources and keeping track of where is what.
- **Job Advertisements Evaluation:** Does the job posting fit my previous experience and skill set? Do I have the years of experience required? Is it in the right location? What languages do I need to speak? Does the salary range meet my needs? When is the role's start date? Those are some of the questions that arise when evaluating a job advertisement's fit to one's requirements.
- **Organization Evaluation:** A common step during the job screening is researching the organization. Some common criteria for evaluating an organization are employee reviews, market performance, reputation, etc.. More benchmarks or criteria can be introduced depending on the market the organization operates in. If you're searching for cross-market opportunities this step can be very time-consuming considering all the factors that need to be taken into consideration.
- **Job Shortlisting:** Before coming to a final decision and after screening tens and hundreds of job listings you decide to shortlist the few that made it through. Whichever approach one chooses to take, will require some kind of rating strategy based on some criteria to sort the listings by. These are subjective ratings based on one's preferences and priorities. You can imagine the amount of mixing and matching between job listings required to complete this task.

> ⚠️ **Note:** The challenges that one experiences during searching for a job are much greater than the ones mentioned in the list above. The list discussed is directed towards the article content and solution scope.

The challenges listed incorporate **repetitive tasks**, **require strong reasoning capabilities**, and a **large amount of content to digest** to arrive at a clear actionable list.

These tasks perfectly meet the key characteristics of employing **LLM-powered agents.**

AI Agent's crucial role in optimizing the job search process (The What..)



Agents workflows — Image by Author

Imagine the end result of hiring 100s of individuals who can screen job advertisements, deeply evaluate the advertising companies and their job postings, and finally tailor their recommendations that best fit your needs. A full end-2-end process that's built on automation and reasoning that significantly reduces the amount of time anyone would ever spend screening online job advertisements.

Let's analyze AI agent's capabilities in terms of automating and personalizing job searches and recommendations. We will further dive into building the engine in subsequent sections. For now, we focus on **What** AI agents bring to the table.

1. Agents can use tools

Agents are capable of taking advantage of defined tools to fulfill their tasks. This involves tools enabling better research capabilities, such as providing access to search the internet and request data using pre-defined APIs. This extends the capabilities of agents towards interfacing with a large selection of tools far beyond the data they were trained on and additionally provides the ability for users to orchestrate how agents solve specific tasks with a selected toolkit.

2. Agents can digest a substantial amount of information

With the increasing size of context windows (1ml+) for AI models, and even reaching infinite context windows, the amount of information agents can digest and reason about is ever-growing. Distributed correctly, agents can analyze an infinite amount of information and provide users with a final summarized version.

3. Agents can reason

While there can be limitations to the reasoning capabilities of AI Agents, the new, improved, and optimized models being published more & more often are closing in on that gap. Considering the task at hand of analyzing job advertisements and comparing those to a user query based on some parameters, the task is potentially considered a straightforward analysis task for a highly trained AI agent.

4. Agents can summarize and structure results

A Tool that can process a large amount of information is valuable but loses its value when it is incapable of delivering that information in an efficient and structured manner. AI Agents are suspectable to hallucinations which can impact how information is summarized and reported. Providing a well-engineered prompt and additional validation steps ensures that Agents fulfill their tasks with correct and reliable results.

5. Agents can collaborate

The difference in background, roles, and tasks agents are assigned to work with, provide a layer of variability that supports approaching problem-solving from different perspectives. Collaboration between agents utilizes this capability to optimize the final results delivered to users.

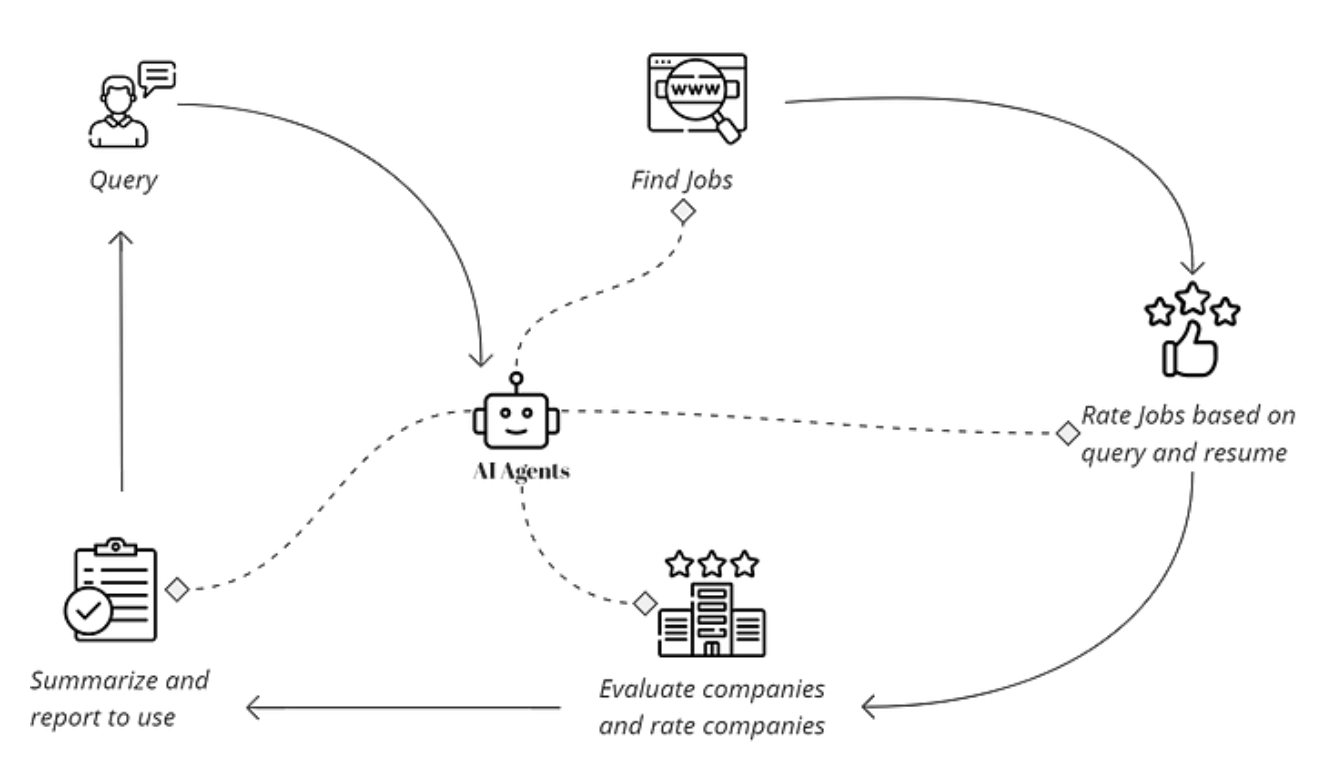
6. Agents are scalable

Dynamic allocation of large amounts of agents to parallel execution of tasks is extremely powerful. The approach shines when problems can be divided into smaller chunks and worked on by separate agents. A large number of agents can be allocated to a batch of job postings, each responsible for tackling a step in the process and passing the result to the next for the next phase. Finally, agents can gather the results for a final summary and structure. That's one approach one can take to build a fast and scalable distributed handling of job advertisements across multiple channels or sources.

How can we put together all those capabilities for our use case?

Introducing AI Agents capabilities to the process (The How..)

To ensure a solution that utilizes all AI capabilities, we introduce agents to every step of the process.


Query
Find Jobs
AI Agents
Rate Jobs based on query and resume
Summarize and report to use
Evaluate companies and rate companies

Agents Tasks — Image Generated by Author

1. Agents will use tools to **retrieve** information about job advertisements. The initial job posting will be retrieved based on the user query.
2. Agents will provide a **rating** and reasoning behind the rating for the job postings based on the user's resume and query.
3. Agents will use internet access tools to **research** information about the source organization for the job advertisement and based on that provide the organizations with a rating score.
4. Agents will finally **summarize** the results and **structure** the results based on a predefined model.

The next section will dive into utilizing the listed AI Agents' capabilities for building a job search engine.

Hands-on Tutorial: Building an AI-Powered Job Search Engine

In the current section, we will walk through the complete implementation step by step and end with analyzing the final results and potential improvements.

Project Structure

The project structure ensures a clear separation between functionalities and components, providing an easier approach to modifying and adapting the code to your requirements. It consists of:

- A `configs` directory that contains all needed configs and parameters to configure agents (role, background, backstory) and tasks (description and expected output)
- A `data` directory containing all needed data to test the job search engine
- A `models` directory containing defined models for the expected output schema
- A `utils` directory containing needed support functions
- `agents_factory.py` and `tasks_factory.py` are used to dynamically generate instances of agents and tasks based on configurations defined.

Copy

```
project/
├── configs
│   └── agents.yml # Agents configuration
│   └── tasks.yml # Tasks configurations
│
├── data
│   ├── sample_jobs.json # JSON file containing job listings
│   └── sample_resume.txt # Text file containing a resume
│
├── models
│   └── models.py #  ORM models
│
├── utils
│   └── utils.py # Utility functions and helpers
│
├── .env # containing all needed environment variables
│
├── agents_factory.py # Factory class for creating agent instances
├── tasks_factory.py # Factory class for creating task instances
│
└── main.py # Main
```

Framework

The project will utilize the crewAI framework to build the end-to-end application. It provides a straightforward interface to build AI Agents with assigned tasks and specific tools. It integrates very well with other available AI frameworks, thus fitting perfectly to the scope of this article.

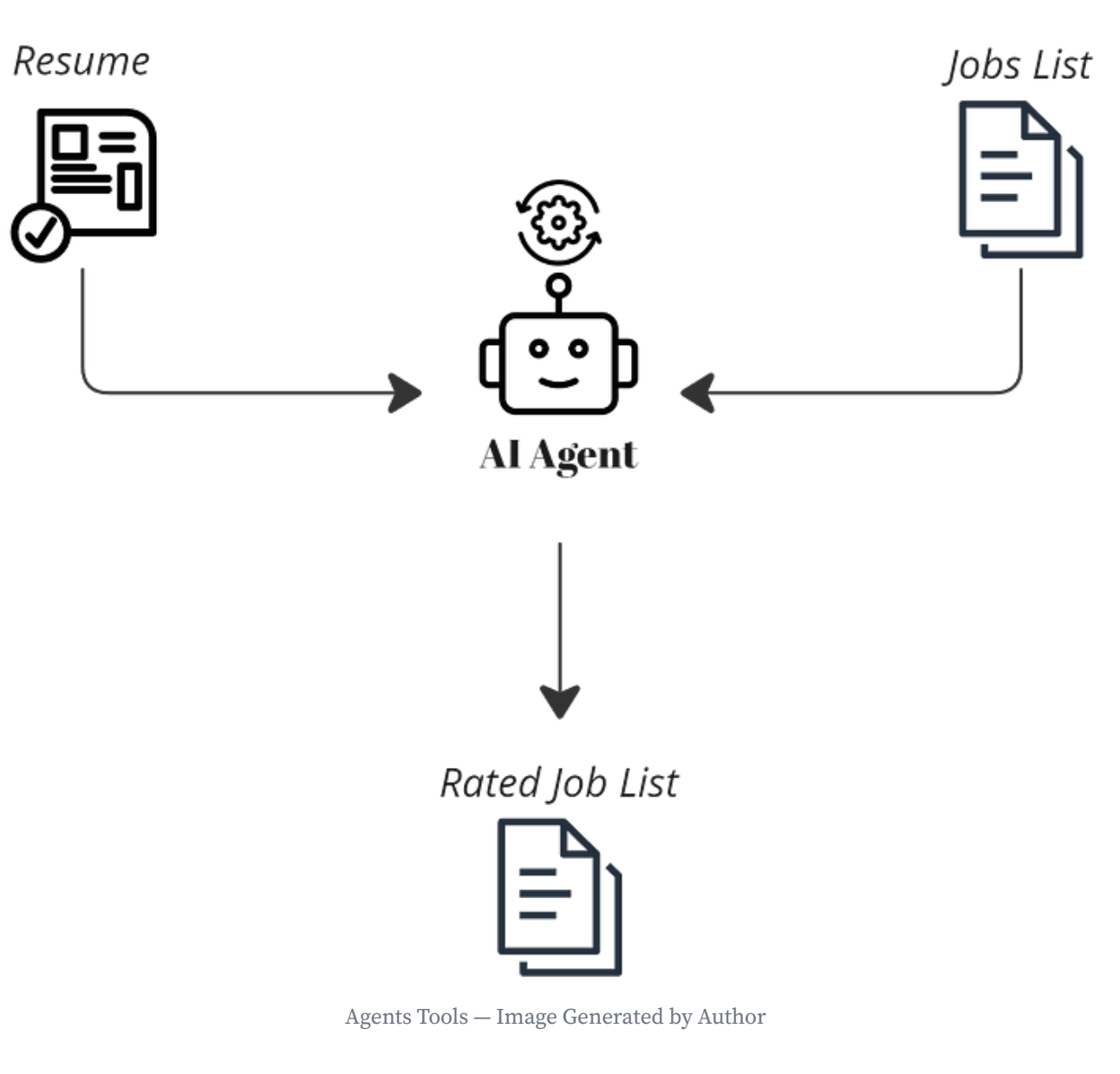
Ensure first that a Python environment (in my case using python-3.11.9) is available and install the framework with all the tools needed.

Copy

```
pip install 'crewai[tools]'
```

Installation of `crewai[tools]` package should cover all the needed packages to run the application.

Tools



Agents Tools — Image Generated by Author

The Agents will require a set of tools to fulfill their assigned tasks. Assignment of tools to agents will depend on how we define the request flow architecture and how information is transmitted from one agent to the other.

Two tools are needed to test our use case, both of which are already available within crewAI.

FileReadTool

On a production scale, where one can manage to accumulate a large list of dependable APIs providing job data, the go-to agent tools are those that can interface with multiple providers and fetch job information based on selected parameters.

For the scope of our application, we will work with an already retrieved JSON response `sample_jobs.json` having a list of job details to keep the focus on showcasing the solution.

A generated resume `sample_resume.txt` is also available to test ratings provided by Agents.

SerperDevTool

Agents will be tasked to gather information about organizations and provide rating feedback to users. This tool supports the process by enabling agents to search the internet. To use the tool Make sure that you create an account on serper.dev to acquire the API KEY needed and load it into the environment.

Ensure that the API key is defined in `.env` file.

Copy

```
SERPER_API_KEY=<>
```

Tools can be directly imported

Copy

```
from crewai_tools import FileReadTool, SerperDevTool
```

Adding tools and assigning them to agents makes extending the application scope with new features very fast and simple.

Data

A `sample_jobs.json` contains a JSON response of a list of sample job advertisements synthetically AI-generated as testing data for our use case.

While the job data presented in the article is in JSON format, it could be easily in any other format for agents to parse, such as personally collected job description any other format for agents to parse, such as personally collected job description data documents stored as simple text, PDF, or Excel files.

Fully utilizing the search process requires agents to use tools to interface with job platforms APIs for full automation and scalability of the data ingestion process. Examples of such official job search API providers are Glassdoor or Jooble.

> 💡 It is **important** that users ensure that collecting any job data adheres to the terms of service set by the providers.

```
{
  "jobs": [
    {
      "id": "VyxlLGIsICxELGUsdixlLGwsbyxwLGUscixELGUsbSxhLG4sdCxTLHkscixhLGM
      "title": "Web Developer",
      "company": "Apple",
      "description": "As a Web Developer at CQ Partners, you will be a leade
      "image": "<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQAkPE
      "location": "Syracuse, NY",
      "employmentType": "Full-time",
      "datePosted": "17 hours ago",
      "salaryRange": "",
      "jobProvider": "LinkedIn",
      "url": "<https://www.linkedin.com/jobs/view/web-developer-at-demant-39
    },
    {
      "id": "VyxlLGIsICxELGUsdixlLGwsbyxwLGUsciwsLCAsVSxYLC8sVSxJLCwsICxCLHI
      "title": "Web Developer, UX/UI, Branding, Graphics",
      "company": "Adobe",
      "description": "Degree required: Bachelor's degree in relevant field..
      "image": "<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSV7-v
      "location": "Columbia, MD",
      "employmentType": "Full-time",
      "datePosted": "1 day ago",
      "salaryRange": "",
      "jobProvider": "LinkedIn",
      "url": "<https://www.linkedin.com/jobs/view/web-developer-ux-ui-brandi
    },
    ......
```

Additionally a `sample_resume.txt` file which contains a generated resume for an experienced data scientist.

You can get the full content of the files on Github.

Tasks

Tasks are distributed to agents equipped with the right tools and skills to complete them.

Each task is defined with a description, an expected output, and an agent responsible for completing it.

```
# configs/tasks.yml
job_search:
  description: |
    find a list jobs that fulfill the following requirements: {query}
  expected_output: A structured output as a valid json with the list of jobs
job_rating:
  description: |
    Using your tools to find the resume file information.
    Provide an additional rating on the jobs you received in context accordi
    The rating is between 1–10 where 10 is the best fit. Every job should ha
    Additionally add a rating_description field that explains the reasoning
    Make sure that all information about the jobs is also maintained in the
  expected_output:  A structured output as a valid json of the list of jobs
evaluate_company:
  description: |
    Using your tools find information about the company of jobs.
    Information can include company culture reviews, company financial repor
    provide an additional rating for the company in a field called company_r
    The rating is between 1–10 where 10 is the best rating. Every job should
    Additionally add a company_rating_description field that explains the re
    Make sure that all information about the jobs is also maintained in the
  expected_output:  A structured output as a valid json of the list of jobs
structure_results:
  description: |
    Use all the context to structure the output as needed for the final repo
  expected_output: A structured output as a valid json of the list of jobs f
```

A `TasksFactory` class is used to generate all needed tasks.

```
# project/tasks_factory.py
from textwrap import dedent
from typing import Optional
from crewai import Agent, Task
from utils.utils import load_config # Load YAML
class TasksFactory:
    def __init__(self, config_path):
        self.config = load_config(config_path)
    def create_task(
        self,
        task_type: str,
        agent: Agent,
        query: Optional[str] = None,
        output_schema: Optional[str] = None,
    ):
        task_config = self.config.get(task_type)
        if not task_config:
            raise ValueError(f"No configuration found for {task_type}")
        description = task_config["description"]
        if "{query}" in description and query is not None:
            description = description.format(query=query)
        expected_output = task_config["expected_output"]
        if "{output_schema}" in expected_output and output_schema is not Non
            expected_output = expected_output.format(output_schema=output_sc
        return Task(
            description=dedent(description),
            expected_output=dedent(expected_output),
            agent=agent,
        )
```

Agents

Agents have a role, goal, and backstory defined to best complete selected tasks.

```
# configs/tasks.yml

job_search_expert:
  role: The best job search Agent
  goal: Impress all requestors with finding the best job postings and advert
  backstory:  The most seasoned job search analyst with lots of expertise in
job_rating_expert:
  role: The best job rating Agent
  goal: Impress all requestors by your ability to find the best fitting job
  backstory: The most seasoned job rating expert with lots of expertise in j
company_rating_expert:
  role: The best company evaluator agent
  goal: find all important information about a company to evaluate job suita
  backstory: The most seasoned company information finder and evaluator expe
summarization_expert:
  role: The best output validator and summarizer
  goal: Making sure that the final output required by the task is fitting th
  backstory: The most seasoned reporting expert that knows how to report on
```

Similar to the `TasksFactory` An `AgentsFactory` class is used to generate all needed AI Agents based on their configuration.

```
# project/agents_factory.py

from typing import Any, List, Optional
from crewai import Agent
from utils.utils import load_config # Load YAML
class AgentsFactory:
    def __init__(self, config_path):
        self.config = load_config(config_path)
    def create_agent(
        self,
        agent_type: str,
        llm: Any,
        tools: Optional[List] = None,
        verbose: bool = True,
        allow_delegation: bool = False,
    ) -> Agent:
        agent_config = self.config.get(agent_type)
        if not agent_config:
            raise ValueError(f"No configuration found for {agent_type}")
        if tools is None:
            tools = []
        return Agent(
            role=agent_config["role"],
            goal=agent_config["goal"],
            backstory=agent_config["backstory"],
            verbose=verbose,
            tools=tools,
            llm=llm,
            allow_delegation=allow_delegation,
        )
```

LLM

The range of options concerning the choice of LLMs is ever-growing, with new models now released almost every week. In this setup, we will be working with Azure Open AI models specifically **gpt-4–32k version 0613**. However, feel free to adapt and test the code to your preferred LLM.

To work with Azure OpenAI models an API KEY and ENDPOINT are needed. Azure provides great documentation to set up and deploy such models.

We add the required environment variables to the `.env` file

```
OPENAI_API_VERSION = <>
AZURE_OPENAI_KEY= <>
AZURE_OPENAI_ENDPOINT = <>
```

Use the code below to import and use the selected model.

```
from langchain_openai import AzureChatOpenAI
import os

azure_llm = AzureChatOpenAI(
            azure_endpoint=os.environ.get("AZURE_OPENAI_ENDPOINT"),
            api_key=os.environ.get("AZURE_OPENAI_KEY"),
            deployment_name="gpt4",
            streaming=True,
            temperature=0 # We set this to 0to ensure a more consitent and ac
        )
```

Output Model

To ensure a final consistent output by agents, we define our desired output models.

```
# models/models.py

from typing import List, Optional
from pydantic import BaseModel
class Job(BaseModel):
    id: Optional[str]
    location: Optional[str]
    title: Optional[str]
    company: Optional[str]
    description: Optional[str]
    jobProvider: Optional[str]
    url: Optional[str]
    rating: Optional[int]
    rating_description: Optional[str]
    company_rating: Optional[int]
    company_rating_description: Optional[str]

class JobResults(BaseModel):
    jobs: Optional[List[Job]]
```

Putting Everything Together: The Crew

The previous diagrams show the sequence of events the agents will work with to achieve the desired output. The completion of one task is handed over to the next Agent optimized with a specific role to handle the task. Thus configuring a sequential process serves our purpose. You can always choose to restructure the design and select a different strategy for tackling the tasks through Agent collaboration and parallel processing.

Finally, we create the search crew within `main.py` to create, assign, and run all AI agents.

```
import json
import os
from textwrap import dedent

from crewai import Crew, Process
from crewai_tools import FileReadTool, SerperDevTool
from dotenv import load_dotenv
from langchain_openai import AzureChatOpenAI
from pydantic import ValidationError
from agents_factory import AgentsFactory
from models.models import JobResults
from tasks_factory import TasksFactory
load_dotenv()

class JobSearchCrew:
    def __init__(self, query: str):
        self.query = query
    def run(self):
        # Define the LLM AI Agents will utilize
        azure_llm = AzureChatOpenAI(
            azure_endpoint=os.environ.get("AZURE_OPENAI_ENDPOINT"),
            api_key=os.environ.get("AZURE_OPENAI_KEY"),
            deployment_name="gpt4",
            streaming=True,
            temperature=0,
        )
        # Intialize all tools needed
        resume_file_read_tool = FileReadTool(file_path="data/sample_resume.t
        jobs_file_read_tool = FileReadTool(file_path="data/sample_jobs.json"
        search_tool = SerperDevTool(n_results=5)
        # Create the Agents
        agent_factory = AgentsFactory("configs/agents.yml")
        job_search_expert_agent = agent_factory.create_agent(
            "job_search_expert", tools=[jobs_file_read_tool], llm=azure_llm
        )
        job_rating_expert_agent = agent_factory.create_agent(
            "job_rating_expert", tools=[resume_file_read_tool], llm=azure_ll
        )
        company_rating_expert_agent = agent_factory.create_agent(
            "company_rating_expert", tools=[search_tool], llm=azure_llm
        )
        summarization_expert_agent = agent_factory.create_agent(
            "summarization_expert", tools=None, llm=azure_llm
        )
        # Response model schema
        response_schema = json.dumps(JobResults.model_json_schema(), indent=
        # Create the Tasks
        tasks_factory = TasksFactory("configs/tasks.yml")
        job_search_task = tasks_factory.create_task(
            "job_search", job_search_expert_agent, query=self.query
        )
        job_rating_task = tasks_factory.create_task(
            "job_rating", job_rating_expert_agent
        )
        evaluate_company_task = tasks_factory.create_task(
            "evaluate_company",
            company_rating_expert_agent,
            output_schema=response_schema,
        )
        structure_results_task = tasks_factory.create_task(
            "structure_results",
            summarization_expert_agent,
            output_schema=response_schema,
        )
        # Assemble the Crew
        crew = Crew(
            agents=[
                job_search_expert_agent,
                job_rating_expert_agent,
                company_rating_expert_agent,
                summarization_expert_agent,
            ],
            tasks=[
                job_search_task,
                job_rating_task,
                evaluate_company_task,
                structure_results_task,
            ],
            verbose=1,
            process=Process.sequential,
        )
        result = crew.kickoff()
        return result

if __name__ == "__main__":
    print("## Welcome to Job Search Crew")
    print("-------------------------------")
    query = input(
        dedent("""
      Provide the list of characteristics for the job you are looking for:
    """)
    )
    crew = JobSearchCrew(query)
    result = crew.run()
    print("Validating final result..")
    try:
        validated_result = JobResults.model_validate_json(result)
    except ValidationError as e:
        print(e.json())
        print("Data output validation error, trying again...")
    print("\n\n########################")
    print("## RESULT ")
    print("########################\n")
    print(result)
```

After task execution, we run a validation check to ensure that the JSON output is validated with the defined model schema. Additionally, we can potentially add further validation loops to ensure agents return the structure needed. We won't go into the details of implementing this complete process, nonetheless if interested feel free to check this great article on the topic.

Result Analysis

We simply run the code

```
python main.py
```

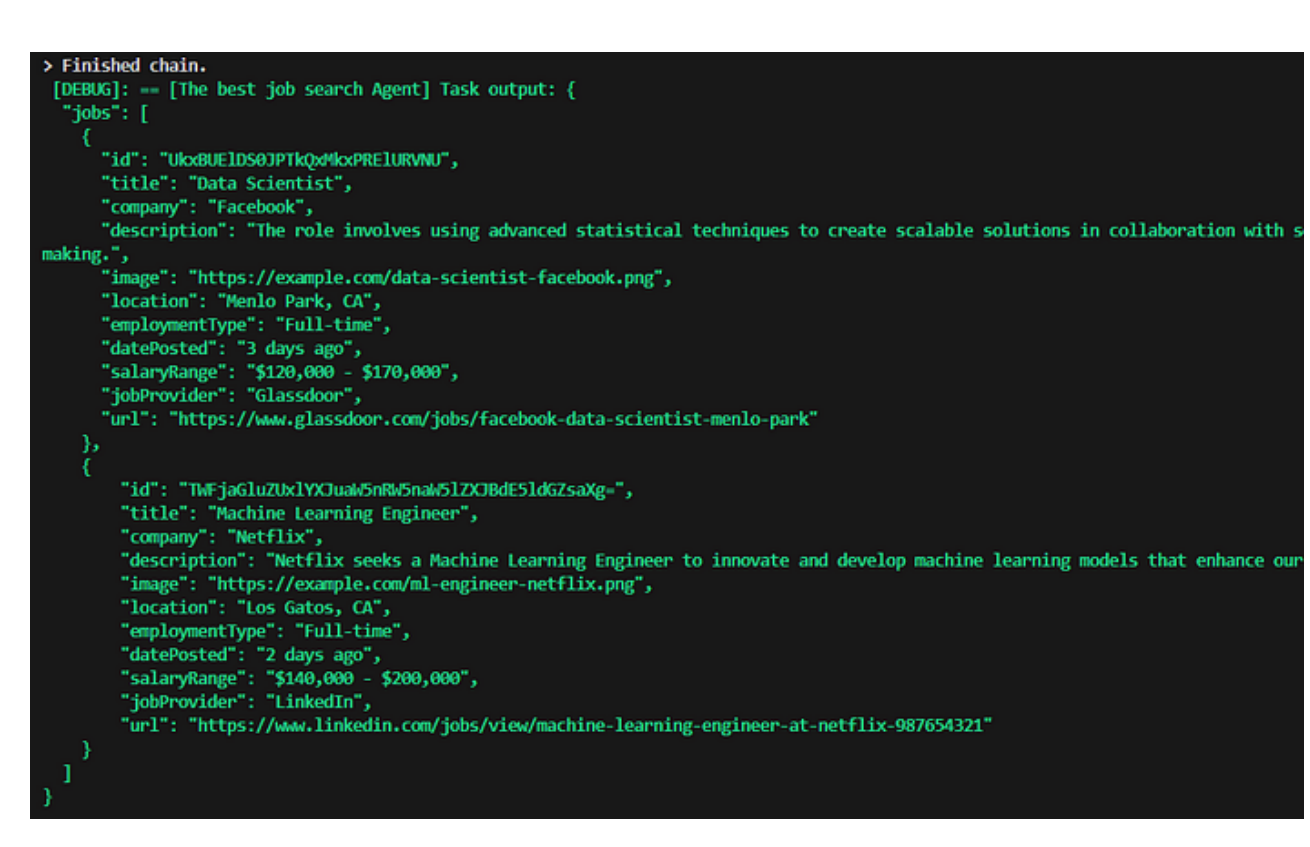
For our initial agent query, we input the following

```
Provide the list of characteristics for the job you are looking for:
A machine learning and data science job in the united states with a salary ra
```

The query can be extended to further include locations, start date, industry etc.. and many other parameters one sees fit to filter and find job listings.

Output:

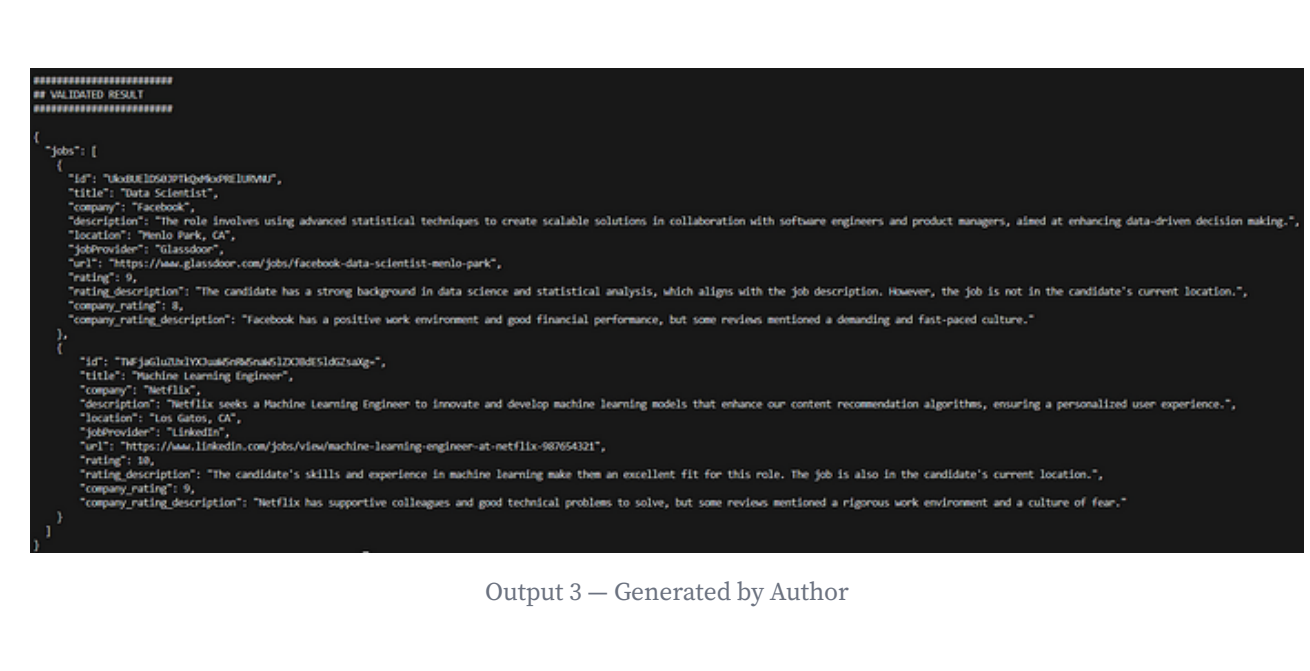
The terminal result shows that the first agent executed successfully and used the tools to retrieve the job list content.



```
## Welcome to Job Search Crew
------------------------------

Provide the list of characteristics for the job you are looking for:
A machine learning and data science job in the united states with a salary range of $100K- $170K
 [DEBUG]: == Working Agent: The best job search Agent


> Entering new CrewAgentExecutor chain...
I need to read the content of the file to find the jobs that match the given criteria.

Action:
Read a file's content

Action Input:
{
  "file_path": "data/sample_jobs.json"
}
```

Output 1 — Generated by Author

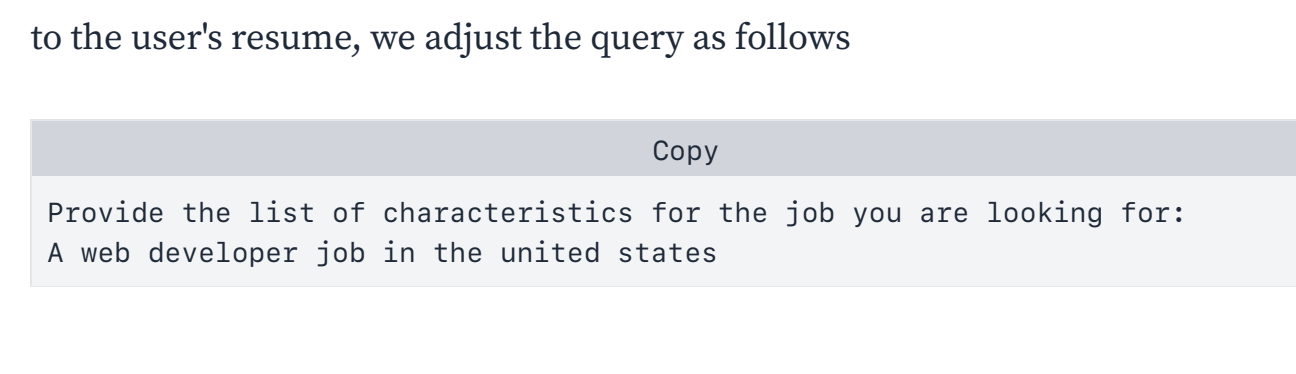
The agent then manages to filter the list and find the correct jobs based on the input query



```
> Finished chain.
 [DEBUG]: == [The best job search Agent] Task output: {
  "jobs": [
    {
      "id": "UkxBUElDS0JPTkQxMkxPRElURVNU",
      "title": "Data Scientist",
      "company": "Facebook",
      "description": "The role involves using advanced statistical techniques to create scalable solutions in collaboration with s
making.",
      "image": "https://example.com/data-scientist-facebook.png",
      "location": "Menlo Park, CA",
      "employmentType": "Full-time",
      "datePosted": "3 days ago",
      "salaryRange": "$120,000 - $170,000",
      "jobProvider": "Glassdoor",
      "url": "https://www.glassdoor.com/jobs/facebook-data-scientist-menlo-park"
    },
    {
        "id": "TWFjaGluZUxlYXJuaW5nRW5naW5lZXJBdE5ldGZsaXg=",
        "title": "Machine Learning Engineer",
        "company": "Netflix",
        "description": "Netflix seeks a Machine Learning Engineer to innovate and develop machine learning models that enhance our
        "image": "https://example.com/ml-engineer-netflix.png",
        "location": "Los Gatos, CA",
        "employmentType": "Full-time",
        "datePosted": "2 days ago",
        "salaryRange": "$140,000 - $200,000",
        "jobProvider": "LinkedIn",
        "url": "https://www.linkedin.com/jobs/view/machine-learning-engineer-at-netflix-987654321"
    }
  ]
}
```

Output 2 — Generated by Author

I will skip the detailed debugging information showing the Agent's progress and jump to the final result.



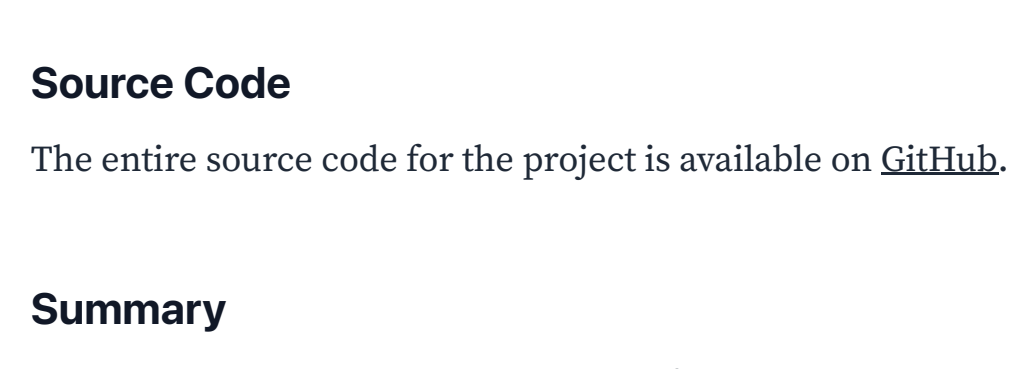
```
########################
## VALIDATED RESULT
########################

{
  "jobs": [
    {
      "id": "UkxBUElDS0JPTkQxMkxPRElURVNU",
      "title": "Data Scientist",
      "company": "Facebook",
      "description": "The role involves using advanced statistical techniques to create scalable solutions in collaboration with software engineers and product managers, aimed at enhancing data-driven decision making.",
      "location": "Menlo Park, CA",
      "jobProvider": "Glassdoor",
      "url": "https://www.glassdoor.com/jobs/facebook-data-scientist-menlo-park",
      "rating": 9,
      "rating_description": "The candidate has a strong background in data science and statistical analysis, which aligns with the job description. However, the job is not in the candidate's current location.",
      "company_rating": 8,
      "company_rating_description": "Facebook has a positive work environment and good financial performance, but some reviews mentioned a demanding and fast-paced culture."
    },
    {
      "id": "TWFjaGluZUxlYXJuaW5nRW5naW5lZXJBdE5ldGZsaXg=",
      "title": "Machine Learning Engineer",
      "company": "Netflix",
      "description": "Netflix seeks a Machine Learning Engineer to innovate and develop machine learning models that enhance our content recommendation algorithms, ensuring a personalized user experience.",
      "location": "Los Gatos, CA",
      "jobProvider": "LinkedIn",
      "url": "https://www.linkedin.com/jobs/view/machine-learning-engineer-at-netflix-987654321",
      "rating": 10,
      "rating_description": "The candidate's skills and experience in machine learning make them an excellent fit for this role. The job is also in the candidate's current location.",
      "company_rating": 9,
      "company_rating_description": "Netflix has supportive colleagues and good technical problems to solve, but some reviews mentioned a rigorous work environment and a culture of fear."
    }
  ]
}
```

Output 3 — Generated by Author

The result contains all the fields that were requested within the configured tasks, in addition to a valid final JSON result structure. We also observe the ratings provided and the reasoning behind those ratings as instructed.

Let's further test if those ratings are indeed are reflective of the matching process to the user's resume, we adjust the query as follows

```
Provide the list of characteristics for the job you are looking for:
A web developer job in the united states
```



```
########################
## VALIDATED RESULT
########################

{
  "jobs": [
    {
      "id": "VyxlLGIsICxELGUsdixlLGwsbyxwLGUscixELGUsbSxhLG4sdCxTLHkscixhLGMsdSxzLGUsLCwgLE4=",
      "title": "Web Developer",
      "company": "Apple",
      "description": "Responsibilities include leading the development, structuring, maintenance, and facilitation of websites and web-based applications, ensuring optimal functionality and user experience.",
      "location": "Syracuse, NY",
      "jobProvider": "LinkedIn",
      "url": "https://www.linkedin.com/jobs/view/web-developer-at-demant-3904417702?utm_campaign=google_jobs_apply&utm_source=google_jobs_apply&utm_medium=organic",
      "rating": 2,
      "rating_description": "The candidate's skills and experience are in data science, not web development. This job is not a good fit.",
      "company_rating": 8,
      "company_rating_description": "Apple has a vibrant and motivating work environment with a culture of creativity and excellence, but it can also be demanding. Its financial performance is strong."
    },
```

We observe that the ratings indeed were reduced with a request for a web developer role while having a resume optimized for machine learning and data science positions.

> 💡 *Feel free to further test the application and optimize it needed to fit your search.*

Source Code

The entire source code for the project is available on GitHub.

Summary

The article discussed the huge impact of introducing AI Agents into the traditional time-consuming job-hunting process. Specifically, the phase of evaluating and matching requirements with online job advertisements. We walked through the key challenges of job searching and positioned AI Agents to carry tasks in seconds that would otherwise take weeks and maybe months for humans to complete. Finally, we implemented the code needed to create and automate AI agents working to find the right personalized jobs for users, utilizing several tools to complete their tasks.

The coming months and years will see AI inevitably being integrated into job search engines and platforms, eventually resulting in a fast and extremely personalized job-matching process. However, having the capability to have control over the design and architecture of your own AI-powered job search engine brings flexibility and an increased advantage to your job search.

Potential Improvements

The use case discussed and implemented in this article represents merely a portion of the potential features that can be added to an AI-powered job search engine. I believe it is inevitable that job search platforms will eventually integrate such tools and leverage their large customer base. What potentially can bring this to a new level is building your own cross-platform solutions that are customized to your needs. Having the flexibility to integrate your chosen models and adapt agents and tasks to your specific requirements.

Some additional features that would be interesting to dive into and build on top

- **Scale** by incorporating a large amount of agents and tools interfacing with multiple job search platforms APIs.
- **Personalize** tasks for adapting and improving resumes and cover letters based on job requirements with an automated application process.
- **Better Preparation** by generating a list of potential interview questions based on job specifications and running mock interviews with agents
- **Improved Salary Negotiation** by running extensive market research for the position and skills advertised.
- **Better Opportunities** through continuous monitoring of advertisements with real-time handling and evaluation.
- ….

> 💡 Happy to hear your suggestions and ideas for further potential improvements !

Enjoyed the Story ?

- Subscribe to get notified when I publish new stories.
- Feel free to reach out to me on **LinkedIn**.

If you're interested in more similar articles check out the following list:

Data Engineering Architectures
Edit description
medium.com

References

U.S. Bureau of Labor Statistics. (2024, April 5). *Table A-12. unemployed persons by duration of unemployment — 2024 Q01 results*. U.S. Bureau of Labor Statistics. https://www.bls.gov/news.release/empsit.t12.htm

Tao, Zhengwei, et al. *A Comprehensive Evaluation on Event Reasoning of Large Language Models*. arXiv:2404.17513, arXiv, 26 Apr. 2024. *arXiv.org*, https://doi.org/10.48550/arXiv.2404.17513.

How many applications does it take to get a job? — movement to work. Movement to Work — Movement to Work is a voluntary collaboration of UK employers committed to tackling youth unemployment through provision of high quality vocational training and work experience opportunities for young people. (2022, July 15). https://movementtowork.com/how-many-applications-does-it-take-to-get-a-job/

crewAI — Platform for Multi AI Agents Systems

#data-science #ai-agent #large-language-models #jobs #artificial-intelligence